

LeetCode 502: IPO

Authored by Arpit Raj, LNMIIT Jaipur | July 13, 2026

Problem Breakdown

- **Objective:** Find the maximized capital after successfully finishing at most K projects.
- **Parameters:** Each project requires an initial capital $l[i]$ and yields a pure profit $l[i]$ upon completion. You start with an initial capital of w .
- **Project Types:**
 - *Not yet affordable:* Required capital $>$ current capital (w).
 - *Affordable:* Required capital $\leq$ current capital (w).
- **Core Strategy:** Out of all currently affordable projects, always pick the one that yields the maximum profit. A Max-Heap is the optimal data structure to efficiently extract this maximum.

Algorithm Approach

- **Step 1:** Pair each project's required capital and resulting profit.
- **Step 2:** Sort the list of projects based on their capital requirements in ascending order.
- **Step 3:** Maintain your current capital pool (w).
- **Step 4:** Iterate through the sorted projects and push every project that has become affordable into a max-heap ($maxh$).
- **Step 5:** Choose the most profitable project available (the top element of the max-heap), add its profit to your current capital, and pop it. Repeat until K projects are finished or no more projects are affordable.

C++ Implementation

```
class Solution {
public:
    int findMaximizedCapital(int k, int w, vector<int>& profits, vector<int>& capital) {

        int n = profits.size();
        vector<pair<int, int>> proj;
        for(int i = 0; i < n; i++) {
            proj.push_back({capital[i], profits[i]});
        }

        sort(proj.begin(), proj.end());

        priority_queue<int> maxh;

        int i = 0; // initialize outside because same project cant be done twice
        while(k-- > 0) {

            while(i < n && proj[i].first <= w) {
                maxh.push(proj[i].second); // taking affordable projects ka profit und
                i++;
            }

            if(maxh.empty()) {
                break;
            }
            w += maxh.top();
            maxh.pop();
        }
        return w;
    }
};
```